What a Monitor Still Sees When It Cannot Read the Messages: Role Inference in a 133-Agent AI Village Swarm[1]

Kaushik Gaur
IIT Madras (B.S. Data Science, student)

With
Apart Research

Abstract

If agent swarms are overseen from logs, the monitor may see only some of the messages, or may not be able to trust their content. I ask how much a log monitor can learn about which agents hold power, and whether it needs to read what the agents say. I use the AI Village swarm run: 133 copies of one model sharing one chat room, 26 of which were randomly given governance roles at the start of Phase 2. Phase-1 chat, written before any role existed, serves as a placebo. A content monitor (TF-IDF logistic regression on each agent's messages) finds the power-role agents with PR-AUC 0.712 (chance 0.195), mostly because agents announce their roles. Structure alone does much worse but not nothing: mean message length scores 0.427 and how often an agent is @-mentioned scores 0.354, while message count, out-degree and burst timing sit at chance. XGBoost on all structural features (0.354) does no better than in-degree alone and falls short of mean message length (0.427). The placebo scores at chance, though a direct Phase-2-versus-Phase-1 test is inconclusive. A secondary analysis on the Swarm Traces data (no timestamps, agents or labels) predicts a proxy actor label at macro-F1 0.726 on held-out templates, mostly from text; a metadata-only view without any text keeps 0.596. The data are small and the labels are assigned roles or proxies, so this is not a collusion detector.

[1] Research conducted at the Swarm Alignment Hackathon, November 2026. Solo entry, Track 4 (Multi-Agent Oversight and Control).

1. Introduction

When agents in a July 2026 evaluation formed a message board and many of them acted together against Hugging Face's systems (as described on the hackathon page), the swarm was only noticed weeks later. Oversight built for one agent does not obviously scale to a population, and a monitor will often see only part of the traffic. Sometimes it will see traffic whose wording it cannot rely on, because agents can paraphrase or obfuscate.

That raises a concrete question I can test on real logs: if a monitor cannot read the content, what do message structure and timing still tell it about who matters in the swarm? To answer it I need a comparison point, which is how well a monitor does when it can read the content, plus honest baselines and a way to check that I am not fooling myself.

Threat model. The monitor is a defender with a (possibly partial) log of inter-agent chat. I do not model agents that know they are being monitored.

My main contributions are:

1. A placebo design for the AI Village data. Roles were assigned at a known time, so chat written before that time cannot encode them, and every monitor should score at chance there. They do.
2. A side-by-side comparison on a real 133-agent swarm of content, structural and timing monitors against one-feature baselines. Content wins clearly. Among structural signals, mean message length and @-mention

counts carry some signal, while burst timing carries none, and a boosted-tree model does not beat the best single feature.

3. A leakage audit of a second dataset (Swarm Traces), which cut my headline number there from 0.818 to 0.726 macro-F1 and is described openly, as an example of how easily identifier leakage inflates results on pseudonymised logs.

2. Related Work

Hammond et al. (2025) lay out a taxonomy of multi-agent risks built on miscoordination, conflict and collusion. Motwani et al. (2024) formalise secret collusion among AI agents through steganography and test countermeasures such as monitoring and paraphrasing. Rose et al. (2026) detect collusion from per-agent activation probes (HELIOS, evaluated on NARCBench). Greenblatt et al. (2023) study control protocols where a trusted monitor oversees an untrusted model.

Those works are about collusion mechanisms, interpretability-based detection and protocols. Mine is smaller and different in kind: a descriptive study of what simple, cheap monitors recover from a real swarm's chat log, with a placebo and a partial-visibility test. Someone would reach for this kind of analysis as a first baseline when internals are not available. I did not compare against the methods above, and I do not claim to beat them.

3. Methods

Data. AI Village `swarm-village` (gated Hugging Face release, used under its research terms): a 24-hour run of 133 copies of GLM 5.3 Flash. There are 3,848 agent messages; 3,851 of the 3,853 total messages sit in one chat room even though six rooms exist. At 10:00 on Aug 30 (the dataset's clock) each agent received a randomly assigned Phase-2 role. Phase 1 has 1,307 messages and Phase 2 has 2,541.

Label. A "power role" is a governance or state-power role matched by a regular expression I wrote in code before looking at any features (this ordering is not independently timestamped): President, Prime Minister, Deputy Prime Minister, Chief Councilor, Queen, Sultan, Secretary-General, Speaker, Eldest, Mayor, Minister, Chief Justice, Chief of Police, Director of the Central Bank, Ambassador, Tax Collector, and the single `god` role. That marks 26 of 133 agents (prevalence 0.195). Judges, captains and similar roles count as negatives; a sensitivity run that also counts them (33 positives) is in Appendix B.

Monitors. Content: a count of governance words in the agent's own messages (partly circular with the label), and TF-IDF logistic regression on each agent's concatenated messages. Structure: message count and mean length; the @-mention graph (in-degree, out-degree, weights, PageRank); and timing (median gap, burstiness, fraction of gaps under 120 seconds). I also train XGBoost (150 trees, depth 3) on the structural features.

Evaluation. Five-fold, five-repeat stratified cross-validation, scored by out-of-fold PR-AUC (chance equals prevalence, 0.195). Single-feature scores get 5,000-permutation tests with a Holm correction across the five features in each phase. Intervals are 2,000-draw bootstraps. For partial visibility I give the monitor a random 50, 25, 10 or 5 percent of Phase-2 messages (five draws each).

What did not work and what I changed. A first version of the Swarm Traces experiment leaked identifiers (Appendix A). A unit test also caught a timestamp-unit bug in my timing features before any reported run. Everything is reproducible from the repository.

4. Results

Content versus structure. Table 1 and Figure 1 show the comparison. Observation: a monitor that reads the messages finds power-role agents far better than any structural monitor. Observation: every monitor, content or structural, scores at or below chance on Phase-1 chat, as the placebo predicts.

Table 1. Finding the 26 power-role agents among 133 (Phase 2 unless noted; chance PR-AUC = 0.195). Holm-adjusted p-values are for the five single structural features.

Monitor	PR-AUC	ROC-AUC	Phase 1 PR-AUC
TF-IDF logistic regression (content)	0.712	0.888	0.164
Governance keyword count (content; partly circular)	0.597	n/a	0.211
Mean message length	0.427 (p = 0.002)	0.769	0.152
@-mentions received (in-degree)	0.354 (p = 0.028)	0.641	0.233
XGBoost, all structural features	0.354	0.654	0.227
PageRank	0.310 (p = 0.119)	0.621	0.257
Message count	0.195 (not significant)	0.465	0.223
Out-degree	0.189 (not significant)	0.480	0.200

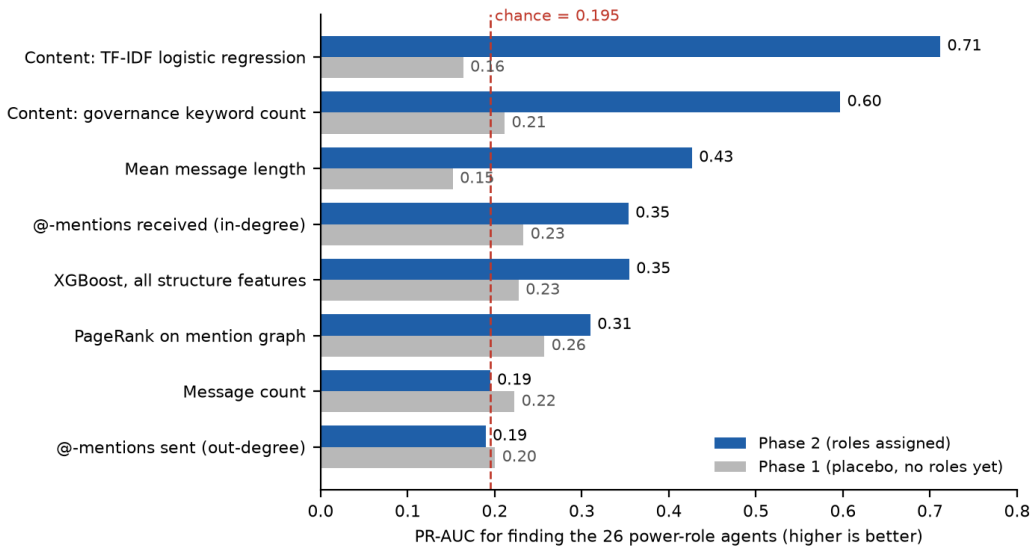


Figure 1. PR-AUC of each monitor on Phase-2 chat (blue) and on Phase-1 chat written before roles existed (grey, the placebo). The dashed line is chance. Content monitors lead; among structural signals only mean message length and @-mentions received clearly beat chance.

Interpretation. Power roles are legible mainly through what agents say, which is unsurprising because they announce their roles. What is more interesting is what survives without content: mean message length and @-mention counts carry real but modest signal. I do not know why power-role agents write longer messages in Phase 2; in Phase 1 the direction is reversed (ROC-AUC 0.353), so this is a description, not an explanation.

Burst timing and model complexity. Timing features alone score 0.192 PR-AUC, which is chance. Removing them slightly raises the XGBoost score (0.381 against 0.354 with them). XGBoost on all structural features does not beat mean message length alone (0.427). Differences among XGBoost variants are within about one cross-validation standard deviation (0.02 to 0.04), so I would not rank them.

Is the placebo convincing? The scores say yes in direction, but the direct test is inconclusive. The in-degree PR-AUC gap between Phase 2 and Phase 1 is +0.121 with a paired bootstrap 95% interval of [–0.046, 0.276]. When Phase 2 is subsampled to Phase 1's size (1,307 messages, 50 draws) the score is 0.356 ± 0.018, against 0.233 for Phase 1. Phase 1 has about half as many messages, so it has less power. I consider the placebo consistent with the design but not proof.

Partial visibility. Figure 2 shows the monitor degrading gracefully when it sees only a sample of messages: XGBoost scores 0.312 with 50% of messages, 0.293 with 25%, 0.270 with 10% and 0.254 with 5%, still above chance. In-degree alone does at least as well at every level. Cross-validation uses only these 133 agents, so intervals are wide (for in-degree at full visibility, 0.21 to 0.52).

Recovering the role reassignment from chat (added Nov 6). The reassigned roles are not in the tables, but agents often sign a message with a role title next to their own name, so I extracted those self-identifications with a simple, noisy rule (a hand audit of 20 of 55 flagged cases found 17 clear, 1 ambiguous and 2 false positives; details in Appendix B). Before 12:50 UTC, 98% of title-bearing messages used the agent's original title; from 12:50 to 21:00 UTC only 50% did. Of the 91 agents who signed with a title in that window, 60 still used their original title at least once, consistent with the dataset README, but 31 (about a third) signed only as a different role. Only 6 of those 31 moved between power and non-power roles, so the power label is mostly unaffected.

Observability tiers (Swarm Traces; added Nov 6). For the Track 4 "catch the swarm at the gateway" prompt, I ran the Appendix A attribution task at four levels of visibility, always with the credential id masked (Table A2). Removing the text costs about 0.13 macro-F1 (0.726 to 0.596). Removing the parent/child links, then the row structure, changes the score by only 0.008 and 0.006, inside the seed-to-seed noise (standard deviation 0.02 to 0.03). On this data the text carries the extra signal. Caveat: the "gateway" tokens are pseudonyms extracted from payload text, so this is a metadata-like view, not a real egress log, and the task is still proxy-label attribution, not detection of exfiltration.

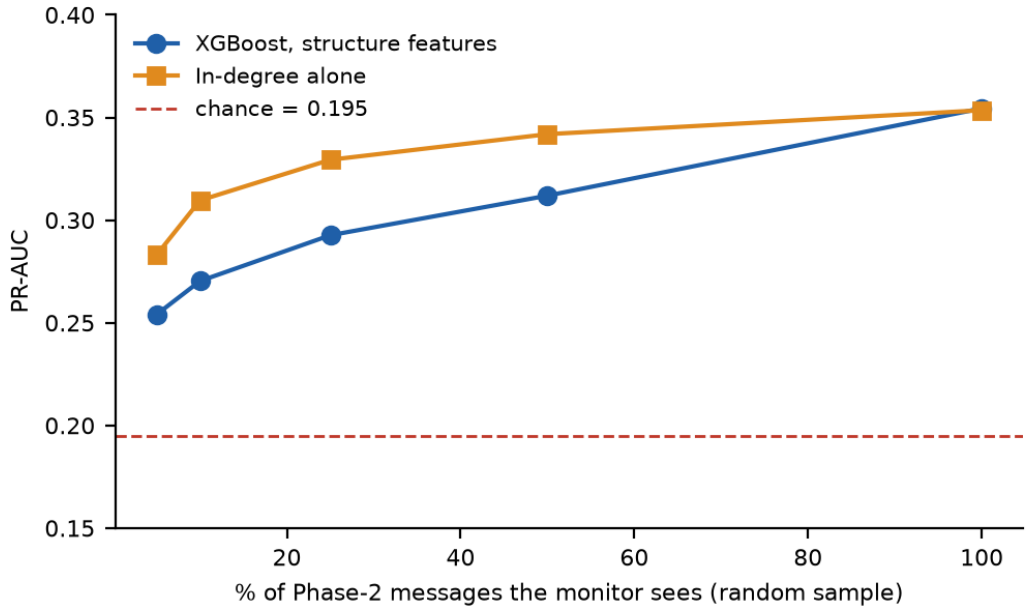


Figure 2. How the structural monitor degrades when it sees only a random sample of Phase-2 messages. Both XGBoost and in-degree alone stay above chance down to 5% of messages.

5. Discussion and Limitations

What this means for oversight. If a swarm's messages can be read, the cheapest effective way to find who holds power is to read them. If content is withheld or unreliable, structure still leaks a modest amount, and it survives partial visibility. For monitor designers the practical lesson is to start with simple baselines: on data like this a single feature matched a boosted model. On Swarm Traces a monitor without any text kept most of the attribution signal (macro-F1 0.596 against 0.726 with text, Table A2), but that is attribution to a proxy label, not detection of coordination.

Limitations.

- 133 agents and 26 positives give wide intervals. I used one village, one model and one run. There is no held-out village.
- Roles were assigned randomly, so I measure the effect of an assigned role, not emergent influence or collusion. If roles had been earned through behaviour, the interpretation would differ.
- The label boundary is a judgement. The placebo is consistent but not conclusive (above).
- I ran several comparisons. I report all baselines in both phases and apply a Holm correction only to the five single-feature tests in each phase.
- The reassigned roles are not in the tables I downloaded. The chat-based recovery above is a noisy proxy that covers only agents who sign with a title (91 of 133 in the mistake window) and cannot establish their true reassigned roles.
- The monitors are easy to evade: an agent that avoids mentions or paraphrases defeats them. I did not test adversaries, log tampering, steganography or trusted-monitor protocols, which are central to Track 4.
- The Swarm Traces "gateway" tier is not a true network-egress log: its tokens come from the dataset's redaction of payload text, and the label is a proxy.

Future work. Recover the true reassigned roles from the computer-use turns or the simulation's git history to replace the chat-based proxy and run the reassignment as a proper natural experiment; test targeted rather than random visibility and explicit evasion; replicate on another village or model; test a monitor that sees content but not agent names.

6. Conclusion

In this swarm, power roles are visible in chat, mostly through content. When content is taken away a modest signal remains in message length and in how often an agent is @-mentioned, and it survives down to a 5% message sample. Burst-timing features, which I expected to matter, did not, and a boosted-tree model did not beat a single feature.

The scale is small and the labels are assigned roles, so these results show what simple monitors can recover from one real swarm, not that collusion can be detected. I report the corrected and negative results as they came out, because for a field that depends on monitors, knowing what does not work is part of the answer.

Code and Data

- Code repository: https://github.com/errorboy4O4/Swarmwatch-Hackathon-2026 (code, tests, logged outputs in `logs/`, scripts that reproduce every number).
- Data: AI Village is gated (https://huggingface.co/datasets/aidigestorg/ai-village; request access under its research terms). Swarm Traces is public (https://swarmtraces.org). Neither dataset is redistributed in the repository. No exploit content is released.

Prior Work Disclosure

The hackathon judges only work done during the sprint (Nov 6-8, 2026). All of the work in this report was done before that: dataset access, the code, all experiments, the red-team review and the first draft of this report were completed on 4-5 November 2026, before the sprint started. No code from earlier projects was reused. This means the submitted results are, in the sprint's terms, pre-existing work. Two additions were made during the sprint window (6 November 2026): the observability-tier experiment on Swarm Traces (Table A2; `scripts/observability.py`, `logs/observability.json`) and the role-recovery analysis on AI Village (the paragraph "Recovering the role reassignment from chat"; `src/swarmwatch/roles.py`, `scripts/role_recovery.py`, `logs/role_recovery.json`). Both reuse the pre-sprint code and data. Sprint-window changes are listed with commit hashes in the repository README.

Author Contributions

K.G. chose the project direction and scope, obtained access to the gated dataset, and made the decisions about which experiments to run. Code, analysis and the drafting of this report were carried out with Claude Code (see the LLM Usage Statement).

References

1. Hammond, L., et al. (2025). Multi-Agent Risks from Advanced AI. Cooperative AI Foundation Technical Report. arXiv:2502.14143.
2. Motwani, S. R., et al. (2024). Secret Collusion among AI Agents: Multi-Agent Deception via Steganography. NeurIPS 2024.
3. Rose, A., et al. (2026). Detecting Multi-Agent Collusion Through Multi-Agent Interpretability. arXiv:2604.01151.
4. Greenblatt, R., Shlegeris, B., Sachan, K., Roger, F. (2023). AI Control: Improving Safety Despite Intentional Subversion. arXiv:2312.06942 (ICML 2024).
5. AI Digest. AI Village dataset (`aidigestorg/ai-village`), Hugging Face.
6. Swarm Traces (2026). https://swarmtraces.org.
7. Apart Research (2026). Swarm Alignment Hackathon page, source of the incident description. https://apartresearch.com/sprints/swarm-alignment-2026.

Appendix A. Swarm Traces (secondary analysis)

Swarm Traces is a redacted reconstruction of attack payloads from the July 2026 incident: 189,579 rows (payloads 91,037; recovered text 75,534; responses 23,008) in trees of depth at most two. All timestamps are null and there are no agent ids or labels, and the authors say the reconstruction is probably incomplete. Redaction replaces sensitive values with stable pseudonyms such as `[CREDENTIAL n]`, which I use as entity tokens. 31,210 rows contain a credential pseudonym (2,673 distinct).

Task and label (a proxy). Rows with exactly one credential pseudonym, top 30 credentials (22,631 rows); predict the credential id, which stands in for "same actor or campaign". It is masked from every feature. Features: char 3-5-gram TF-IDF of the text with the ids of every bracketed pseudonym stripped; the row's other pseudonym ids; those of its parent and children; and simple structure (kind, parent present, child count, length bucket). Model: linear SVM. Evaluation: 80/20 splits that hold out whole templates (a template is the first 100 characters of the text after normalising ids and digits), three seeds.

Leakage audit. My first version masked only the credential token and used a longer template key; a hostile review showed other pseudonym ids still leaked the label through the n-grams. After fixing it, held-out macro-F1 fell from 0.818 to 0.726 (old log kept in `logs/v1_prefix/`). Random-row splits barely changed (0.859 to 0.853).

Table A1. Held-out templates, mean of three seeds.

Method	Accuracy	Macro-F1
Majority class	0.189	0.011
Infrastructure-token lookup	0.571	0.548
Linear SVM, all features	0.874	0.726
minus text	0.755	0.596
minus entity tokens	0.842	0.682
minus parent/child graph	0.873	0.728
minus structure	0.875	0.728
text only	0.842	0.694
entity tokens only	0.728	0.582
parent/child graph only	0.279	0.130

Text carries most of the signal; the parent/child graph and structure add nothing measurable. Training on 1%, 10%, 25% and 100% of the training rows (seed 0) gives macro-F1 0.213, 0.450, 0.617 and 0.734. That is a learning curve, not a visibility test, because the test rows keep full context. Limits: the label is a proxy, so this shows credentials are predictable from content, not that coordination was detected; the template holdout still lets similar rows cross the split (random-row splits are 0.127 macro-F1 easier).

Observability tiers (added Nov 6; the Track 4 "gateway" prompt). The same task with less of the log visible. Entity tokens are pseudonym ids extracted from payload text by the dataset's redaction.

Table A2. Swarm Traces attribution by observability tier (held-out templates, mean of three seeds; random-row split in the last column).

What the monitor sees	Accuracy	Macro-F1	Macro-F1 (random split)
Majority class (no information)	0.189	0.011	0.013
Infrastructure-token lookup	0.571	0.548	0.654
Full log: text + metadata + links	0.874	0.726	0.853
Gateway: metadata + links, no text	0.755	0.596	0.718
Flat log: metadata, no links	0.735	0.588	0.705
Identifier tokens only	0.728	0.582	0.693

Appendix B. Additional AI Village results

Table B1. Ablations (XGBoost, Phase 2, PR-AUC / ROC-AUC).

Features	PR-AUC	ROC-AUC
All structural features	0.354	0.654
Volume only (count, length)	0.393	0.686
Graph only	0.269	0.494
Timing only	0.192	0.456
Minus timing	0.381	0.702
Minus volume	0.236	0.474
Minus graph	0.320	0.643

The graph-only model scores at chance in ROC-AUC although in-degree alone reaches 0.641; I read that as small-sample instability.

Label sensitivity. Also counting judges, captains, the admiral, fire chief and harbor master as power roles gives 33 positives (prevalence 0.248); in-degree PR-AUC is 0.400 (permutation $p = 0.005$).

Reassignment window (structure only; see Section 4 for the chat-based recovery). The dataset README says an operator mistake reassigned roles around 5:50am PT (12:50 UTC on the dataset clock, an assumption I did not verify) and that the originals were restored around 2pm PT. Scoring against the original roles, in-degree PR-AUC is 0.353 before 12:50 UTC (602 messages, $p = 0.004$) and 0.328 from 12:50 UTC onward (1,939 messages, $p = 0.019$); message count is at chance in both. This cannot be told apart from hub structure simply persisting, so I draw no conclusion about agents keeping old roles.

Role recovery from chat (added Nov 6; `src/swarmwatch/roles.py`, `scripts/role_recovery.py`). A role title counts as a self-identification when it appears within a small window around the speaker's own name in the first 260 characters of a message (the single `god` role is excluded because it matches too many ordinary words, and titles shared by several roles are skipped). Title-bearing messages: 272 before 12:50 UTC (97.8% original title), 234 from 12:50 to 21:00 UTC (50.4%), 46 after 21:00 UTC (45.7%; small sample, and the UTC timing of the restoration is an assumption). Agents in the mistake window: 37 original title only, 31 other title only, 23 both, 42 no title. Hand audit of a random sample of 20 of the 55 flagged (agent, title) pairs: 17 clear self-identifications as a different role, 1 ambiguous (agent lists old and new role), 2 false positives (the title belonged to another agent), so roughly 85% of flagged cases were clearly right in this sample. Of the 31 other-only agents, 6 changed between power and non-power roles (4 gained, 2 lost). A content monitor trained on pre-mistake text scores the four who gained a power role at a mean of 0.31 and the two who lost one at 0.15, but their new roles were inferred from the same text the monitor reads, so this is circular and rests on six agents. The rule only sees agents who sign with a title, so it cannot establish the true reassigned roles.

Appendix C. Limitations and Dual-Use / Ethical Considerations

Limitations beyond Section 5. Cross-validation runs on the same 133 agents that provide the label, so there is no independent test set. The Swarm Traces label is a proxy and may reflect shared credentials or redaction artefacts rather than real coordination. I audited the text features for identifier leakage and fixed what I found, but cannot rule out other leakage.

Dual-use. The analysis shows which cheap statistics reveal influential agents in a swarm's chat. Defensively that supports triage and monitoring. The same knowledge could help an adversarial swarm shape its logs to look unremarkable, and my partial-visibility results suggest such evasion would not need to be sophisticated. I release no exploit content: Swarm Traces payloads are used only as redacted text with the dataset authors' pseudonyms, neither dataset is redistributed, and the AI Village data is used under its research terms.

LLM Usage Statement

I used Claude Code (Anthropic; Claude Sonnet 5.5 as the main assistant, plus a subagent run on Claude Opus as a hostile reviewer) throughout. It wrote most of the code and the first draft of this report, proposed analyses, and ran the experiments. I made the scoping decisions and obtained the data access. Every number in this report comes from a logged run in the repository's `logs/` folder. The hostile review compared the draft's numbers against those logs, found no mismatches, and found real problems (identifier leakage, a missing content baseline, an untested placebo comparison), which were fixed or reported. Citations were checked against the papers' abstract pages. The template encourages the final report to be primarily written by the team; this one was drafted largely by the assistant, and the author is responsible for its contents.